Singleton, Factory method, Abstract factory

T120B516 Objektinis programų projektavimas

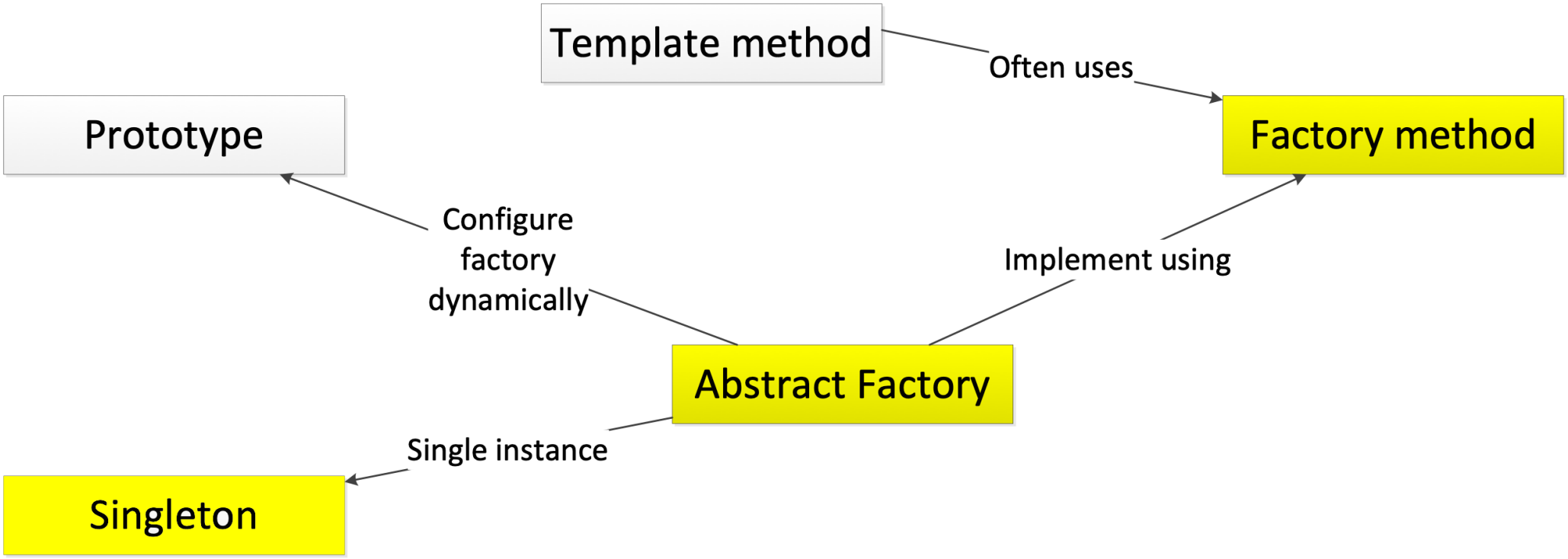
2026 rudens semestras

Dr. Andrej Ušaniov

Šiandien spręsime vieną problemą: kas turi kurti objektus?

- **Reikia tik vieno objekto** → Singleton
- **Nežinau, kurį objektą sukurti** → Factory Method
- **Reikia suderintos objektų šeimos** → Abstract Factory

Design pattern relationships



Singleton

- GoF - “Ensure a class has only one instance and provide a global point of access to it.”
- Want to make sure only one object of a particular class can be created.

- **Ensure that a class has just a single instance**.
- **Provide a global access point to that instance**.

Kurių objektų žaidime iš tikrųjų reikia tik po vieną?

- GameSettings? Score? Player? Logger? Texture? Lobby?
- Kas nutiktų, jei GameSettings turėtume 10 egzempliorių?

Singleton – when to use?

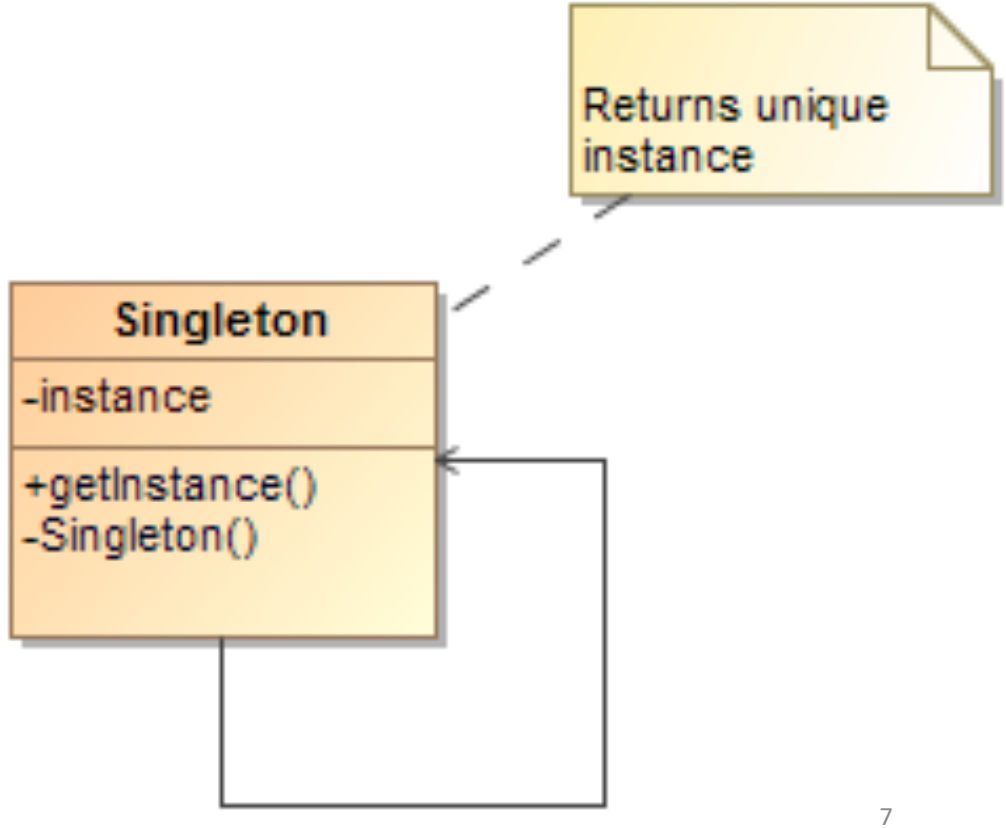
- You use the Singleton design pattern when you want to either **restrict resource use** (instead of creating numbers of large objects without limit) or
- when you have a sensitive object whose **data shouldn't be accessed by multiple instances** (such as a registry).

- Mostly used in multi-threaded and database applications. It is used in **logging, caching, thread pools, configuration settings, load balancer etc**.

Kaip neleisti sukurti antro objekto?

- Private constructor to avoid client applications to use constructor
- `getInstance` or `createInstance` - usual name for that static method
- Suraskite per 10 sekundžių klaidas

Singleton GoF structure



Lazy instantiation

```
public class Singleton {
        private static Singleton instance = null;
        private Singleton() {
                System.out.println("Singleton initialized");
        }
        public static Singleton getInstance() {
                if(instance == null){
                        instance = new Singleton();
                }
                System.out.println(”Returning instance");
                return instance;
        }
}
```

O kas, jei getInstance() vienu metu iškviečia dvi gijos?

- If two threads at the same time would check if instance is null, and no Singleton object exists yet, they could both get past the "**`if`**"
- One easy fix is to restrict access to **`getInstance`** to one thread at a time:
  - **`synchronized`** - Java keyword,
  - **`lock()`** – C# keyword.

Synchronized method Java

```
public class Singleton {
      private static Singleton instance = null;
      private Singleton() {
            System.out.println("Singleton initialized");
      }
      public static synchronized Singleton getInstance() {
            if(instance == null){
                  instance = new Singleton();
            }
            System.out.println(”Returning instance");
            return instance;
      }
}
```

Locked method C#

```
public class Singleton{
      private static Singleton instance = null;
      private object threadLock = new object(); //lock token
      private Singleton() {
            Console.WriteLine("Singleton initialized");
      }
      public static Singleton getInstance(){
            lock(threadLock){//using the lock token
                  if(instance == null){
                        instance = new Singleton();}
            }
            Console.WriteLine("Returning instance");
            return instance;
      }
}
```

Išsprendėme vieną problemą. Bet ką už tai sumokėjome?

Low performance of thread safe synchronized methods

Maintenance of locks slows down significantly due to:

1) Constant monitoring of the entrance to the synchronized method
2) Blocked thread looses time awaiting for its turn (each time you need to collaborate with Singleton)

Infamous Double-Checked Locking (DCL) idiom

```
static volatile Singleton instance;
public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null)
                instance == new Singleton();
        }
    }
return instance;
}
```

// volatile isn't that much faster than synchronized, synchronized is pretty fast nowadays, and DCL requires more
// code, after Java 1.5 came out, you can continue using plain old synchronization.

Eager instantiation (thread safe)

```
public class Singleton {
      private static Singleton instance = new Singleton();
      private Singleton() {
            System.out.println("Singleton initialized");
      }
      public static Singleton getInstance() {
            System.out.println(”Returning instance");
            return instance;
      }
}
```

Before execution of an application, all the classes are loaded first.

And while loading the classes, all their data members are instantiated.

**Java Specification: the class initialization phase is sequential.*

Static initialization solves problems of thread safetiness and low performance

- Don't let **`getInstance`** create any objects at all — let it return only that one object that's already been created when the code is first loaded into the Java Virtual Machine

//JAVA

```
public class Singleton {

    private static final Singleton instance = new
Singleton();

private Singleton(){}

    public static Singleton getInstance(){
        return instance;
    }
}
```

//C#

```
public sealed class Singleton{

   private static readonly Singleton instance = new
Singleton();

   private Singleton(){}

   public static Singleton Instance
   {
      get {return instance;}
   }
}
```

The “Inner class” approach (initialization-on-demand holder pattern)

```
public class Singleton{

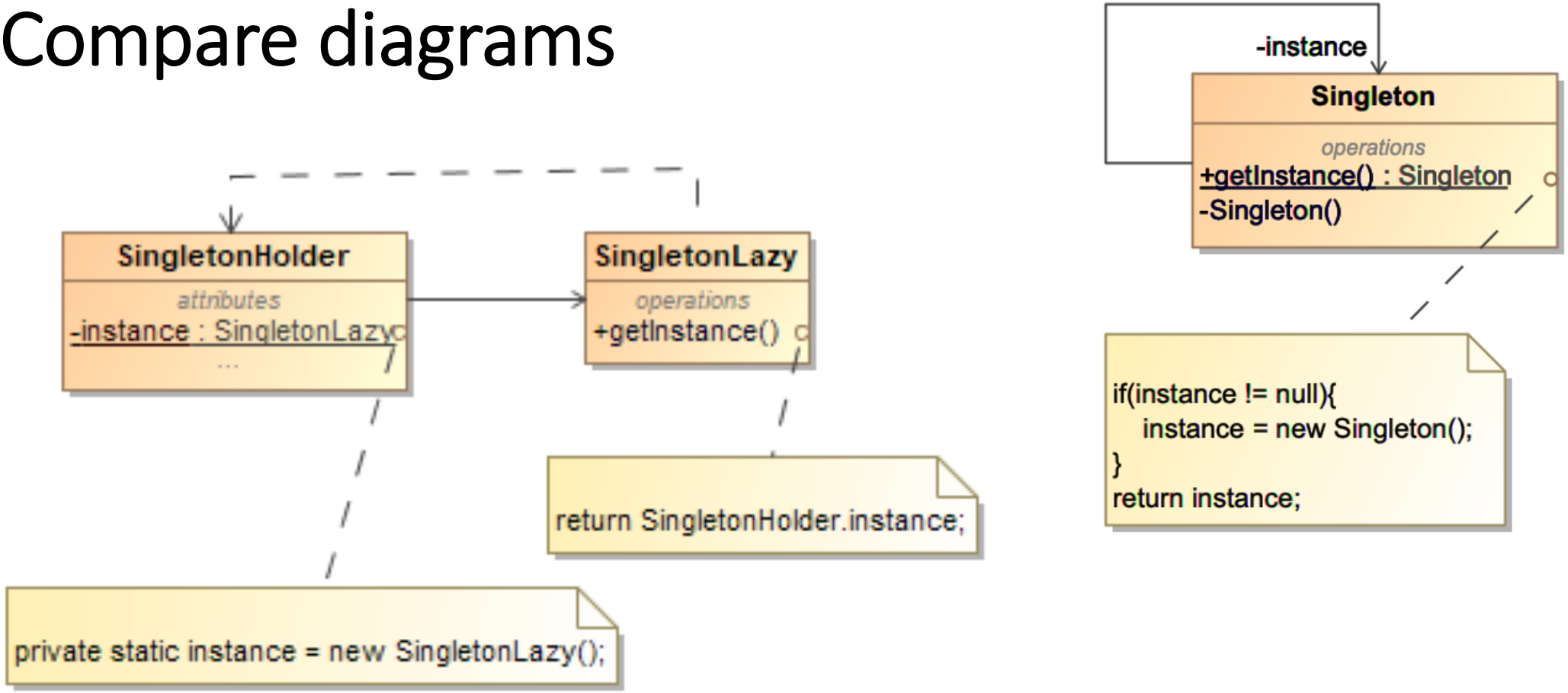
	private static class SingletonHolder {
		static Singleton instance = new Singleton();
	}

	private Singleton() {
		System.out.println("Smart Singleton initialized");
	}

	public static Singleton getInstance() {
		System.out.println("Singleton initialized via Holder");
		return SingletonHolder.instance;
	}
}
```

**Can only be used when the construction of Singleton will not fail.*

Compare diagrams



- *Išbandykite per laboratorinius* ***`SingletonHolder`***

Singleton Pros and Cons

You can be sure that a class has only a single instance.

You gain a global access point to that instance.

The singleton object is initialized only when it's requested for the first time.

Violates the Single Responsibility Principle. The pattern solves **two problems at the time**.

It may be **difficult to unit test** the client code of the Singleton because many test frameworks rely on inheritance when producing mock objects. Since the constructor of the singleton class is private and overriding static methods is impossible in most languages, you will need to think of a creative way to mock the singleton. Or just don't write the tests. Or don't use the Singleton pattern.

The pattern requires **special treatment in a multithreaded environment** so that multiple threads won't create a singleton object several times.

Singleton pattern can mask bad design

- **1. Slaptos priklausomybės -** Klasė per konstruktorių nedeklaruoja, ko jai reikia. Kreipimasis į Singletoną paslepiamas metodų viduje, todėl kitiems neaišku, nuo ko klasė priklauso, ir ją sunku panaudoti pakartotinai.
- **2. Sluoksnių pažeidimas („žino per daug“) -** Kadangi Singletonas pasiekiamas iš bet kur, kyla pagunda jį kviesti žemo lygio komponentuose (pvz., UI mygtuke tiesiogiai kviesti DB). Taip peršokami architektūriniai sluoksniai ir pažeidžiamas Demetros dėsnis (*Law of Demeter*).
- **3. „Visagalio objekto“ (God Object) rizika -** Dėl lengvo pasiekiamumo į Singletoną pradedama kišti viskas: sesijos, konfigūracijos, DB ryšiai ir podėlis. Jis tampa milžinišku objektu, kurio mažiausias pakeitimas gali sulaužyti nepropregnuojamas programos dalis.
- **4. Neįmanomas automatinis testavimas (Unit Testing) -** Singletonas išlaiko globalią būseną visą programos veikimą. Testuojant jo negalima pakeisti netikru objektu (*Mock*), o vienas testas gali pakeisti duomenis ir sugadinti kitų testų rezultatus.
- **Apibendrinimas -** Singletonas užmaskuoja blogą dizainą, nes leidžia greitai gauti duomenis iš bet kur išvengiant teisingo architektūrinio planavimo. Ilgainiui kodas virsta neišpainiojamu mazgu (*spaghetti code*), todėl moderniose sistemose jį pakeitė **Priklausomybių Įliejimas (Dependency Injection)**.

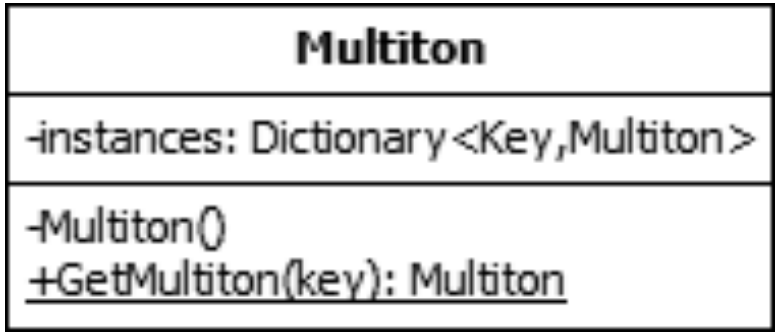
```
UI → Database.Instance.ExecuteRawSql()
```

Singleton - subclassing

- If you’re in a situation where you think you have to start subclassing a singleton, better think twice — is the object you’re working with really a singleton?

Multiton pattern

- Multiton pattern generalizes the singleton pattern. This is collection (Hash, Dictionary or Map) of instances paired with respective key and **ensures a single instance per key**.




```
using System.Collections.Generic;

public enum MultitonType {
    ZERO,
    ONE,
    TWO
};

public class Multiton {
    private static readonly Dictionary<MultitonType, Multiton> instances =
        new Dictionary<MultitonType, Multiton>();
    private int number;

    private Multiton(int number) {
        this.number = number;
    }

    public static Multiton GetInstance(MultitonType type) {
        // lazy init (not thread safe as written)
        // Recommend using Double Check Locking if needing thread safety
        if (!instances.ContainsKey(type)) {
            instances.Add(type, new Multiton((int)type));
        }
        return instances[type];
    }

    public override string ToString() {
        return "My number is " + number.ToString();
    }

    // Sample usage
    public static void Main(string[] args) {
        Multiton m0 = Multiton.GetInstance(MultitonType.ZERO);
        Multiton m1 = Multiton.GetInstance(MultitonType.ONE);
        Multiton m2 = Multiton.GetInstance(MultitonType.TWO);
        System.Console.WriteLine(m0);
        System.Console.WriteLine(m1);
        System.Console.WriteLine(m2);
    }
}
```

Ar tikrai tą patį paveikslėlį reikia krauti 1000 kartų?

- Įkeliant tą patį lėktuvėlio ar kito objekto paveikslėlį daug kartų, nėra prasmės kiekvienam objektui iš naujo krauti ir laikyti identiškus resursus atmintyje.

Visi variantai sukuria vieną objektą. Bet ar visi kainuoja tiek pat?

- Našumo eksperimentas – palyginkime kelias Singleton realizacijas ir matuoji vykdymo laiką bei atminties naudojimą.

Eksperimento rezultatai

```
=== Singleton design pattern time and memory test ===
Initialised memory 12 Mb
1) Lazy Singleton
Lazy Singleton instantiation took: 210ms
Lazy Singleton memory utilized: 5316296 bytes | 5 MB
2) Lazy Thread Safe Singleton
Lazy Thread Safe instantiation took: 151ms
Lazy Singleton memory utilized: 10779376 bytes | 10 MB
3) Eager Singleton
Eager instantiation took: 115ms
Lazy Singleton memory utilized: 8084520 bytes | 7 MB
4) Inner Holder Singleton
First call to InnerHolderSingleton getInstance()
Singleton initialized into Lazy holder
Inner Holder Singleton instantiation took: 110ms
Inner Holder Singleton memory utilized: 8084520 bytes | 7 MB
```

- **Kurią Singleton realizaciją pasirinktumėte žiūrėdami tik į šiuos rezultatus?**

Factory

- Type: Creational
- Also Known As: Factory Method, Virtual Constructor, Creator
- Intent: “Define an interface for **creating an object**, but let subclasses decide **which class** to instantiate. Factory method lets a class defer instantiation to subclasses.”

When to use “Factory”?

- **You do not know ahead** of time what class object you need. The return class is chosen at run time
- **To encapsulate object creation** when you don’t want the client to know about every subclass
- **To centralize class selection** when a method return several classes that share a common superclass

Kas nutiks, jei naujo objekto sukūrimas išmėtytas po visą programą?

- Kai keli žaidimo metodai kuria kliūtis ar kitus objektus, Pasikeitus objekto konstruktoriui reikia surasti visas `new` vietas - Tai labai aiškiai motyvuoja objekto kūrimo centralizavimą.

Ar objekto naudotojas visada turi žinoti, kaip jį teisingai sukurti?

- Vienas programuotojas sukūrė klasę, o kitam reikia žinoti, kokius 5 parametrus jai perduoti ir kaip ją teisingai inicializuoti. Factory paslepia šias kūrimo detales.
- Duok man objektą. Man nesvarbu, kaip jį sukūrei.

Factory method Pros and Cons

You avoid tight coupling between the creator and the concrete products.

Single Responsibility Principle. You can move the product creation code into one place in the program, making the code easier to support.

Open/Closed Principle. You can introduce new types of products into the program without breaking existing client code.

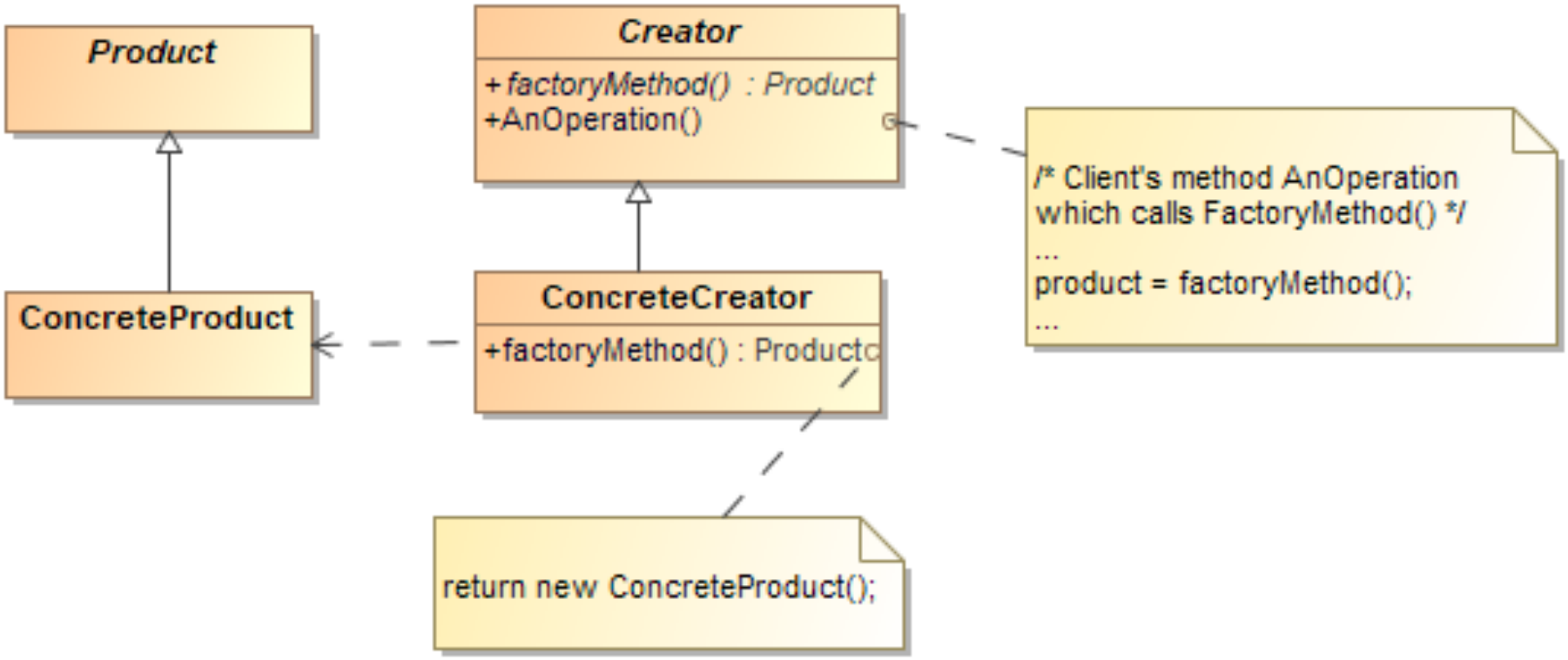
The code may become more complicated since you need to introduce a lot of new subclasses to implement the pattern. The best case scenario is when you're introducing the pattern into an existing hierarchy of creator classes.

Tai kur dabar turėtų gyventi ”new”? (Design insight)

- Vienoje kūrimui skirtoje vietoje.

- Replace direct object construction calls (using the *new* operator) with calls to a special *factory* method
- You might need to add a *temporary parameter* to the factory method to control the type of returned product.

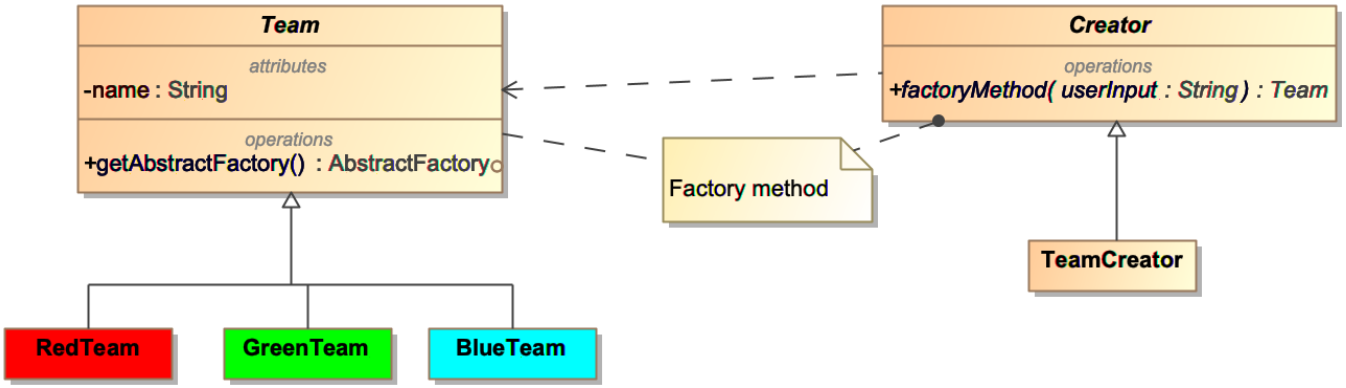
Factory's structure (GoF)



The client shouldn't care about the concrete class of the factory it works with.

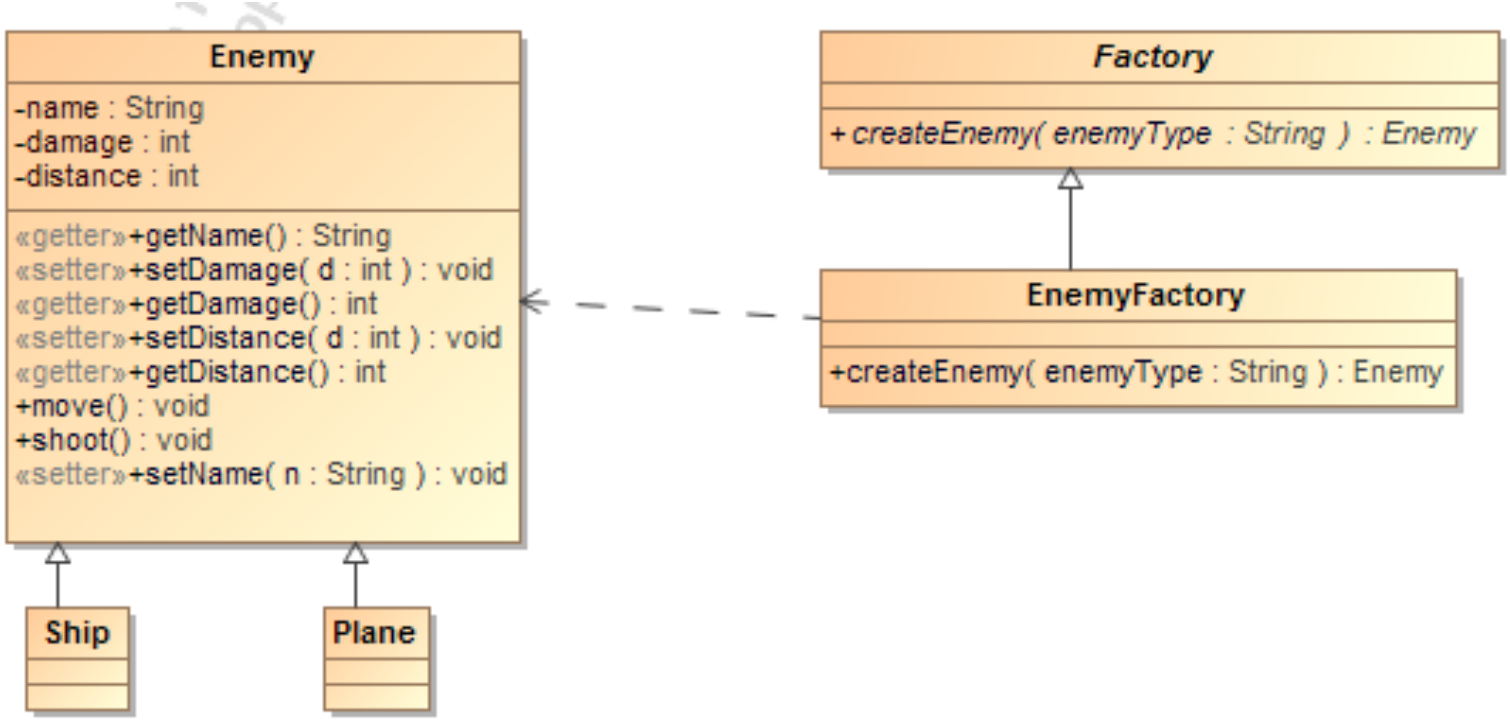
Example to code 1/2

- Different team factory for our game



Example to code 2/2

- Enemies factory



Abstract Factory

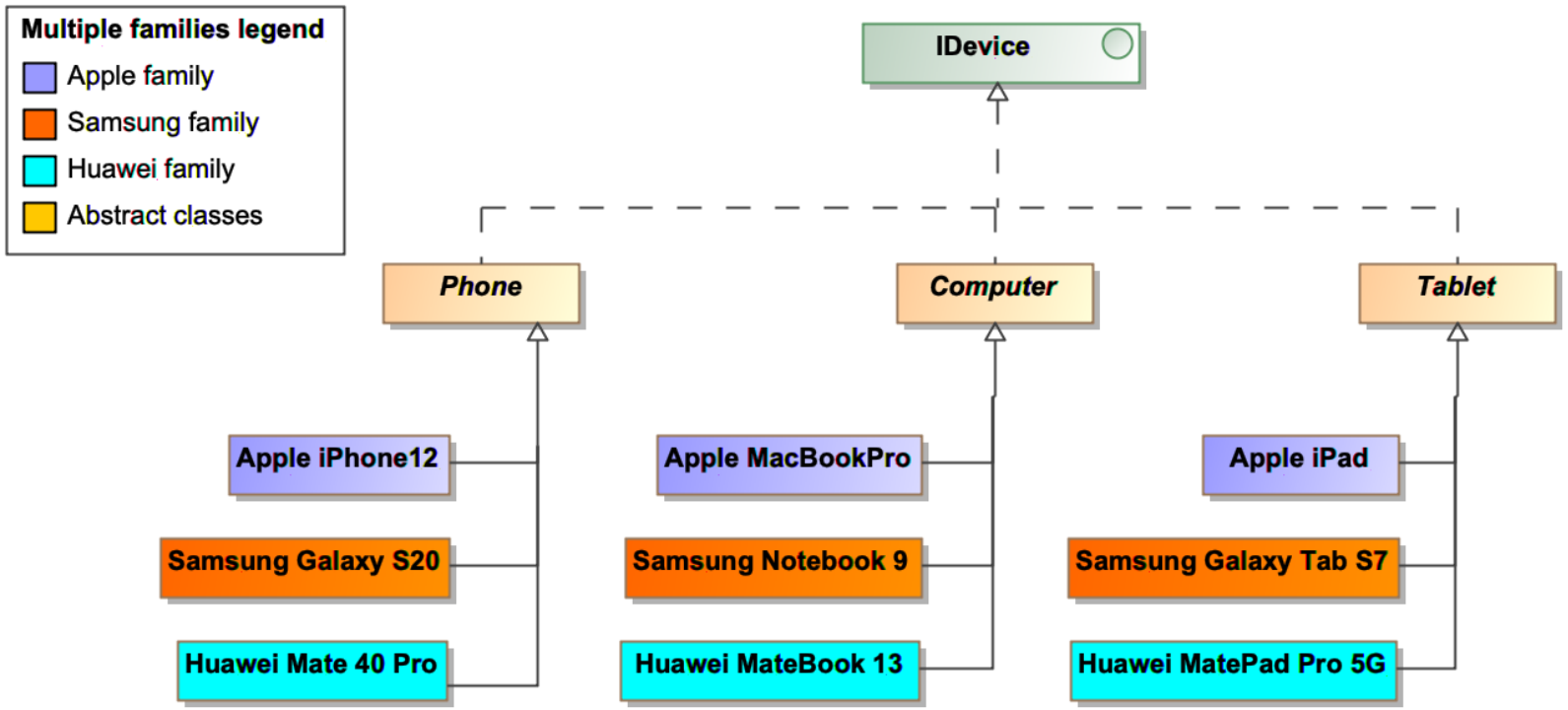
- Type: Creational
- Also Known As: Kit (Widget kit, Dialog Kit for look-and-feel)
- Intent: “Provide an interface for **creating families of related or dependent objects** without specifying their concrete classes”

When to use “Abstract Factory”

- A system should be configured with **one of multiple families of products**
- Sometimes, you might need **more than one factory** to create **objects of a similar nature**.
- A system should be independent of how its products are created, composed and represented

Telefonas, planšetė, kompiuteris... o dabar pridėkime skirtingus gamintojus.

Kas dabar yra produkto tipas, o kas produkto šeima?





O jeigu viena gamykla kurio nors produkto negamina? Klasikinė šablono dilema

- Šablono prielaida yra ta, kad visos gamyklos geba pagaminti visą produktų šeimą, priešingu atveju pažeidžiamas bazinės gamyklos interfeisas (sutartis)
- Sprendimas - Null Object Pattern - nemeta klaidos, o grąžina objektą, kurio metodai tiesiog nieko nedaro. Nereikia tikrinti `!= null` ar gaudyti klaidų. Tinka, kai tai netrukdo sistemos veikimui, kad ausinės tiesiog „tyli“
- Interface'ų atskirimas – bazinis interfesas bendriems produktams, ausinių gamybą iškelti į atskirą sąsają
- metode createHeadphones() grąžinimo tipas apibrėžiamas kaip Optional<Headphones> (Java/C++) arba Headphones?(C#)
- Exception metimas – pažeidžia LSP, verčia naudoti try-catch

Abstract Factory Pros and Cons

You can be sure that the products you're getting from a factory are **compatible** with each other.

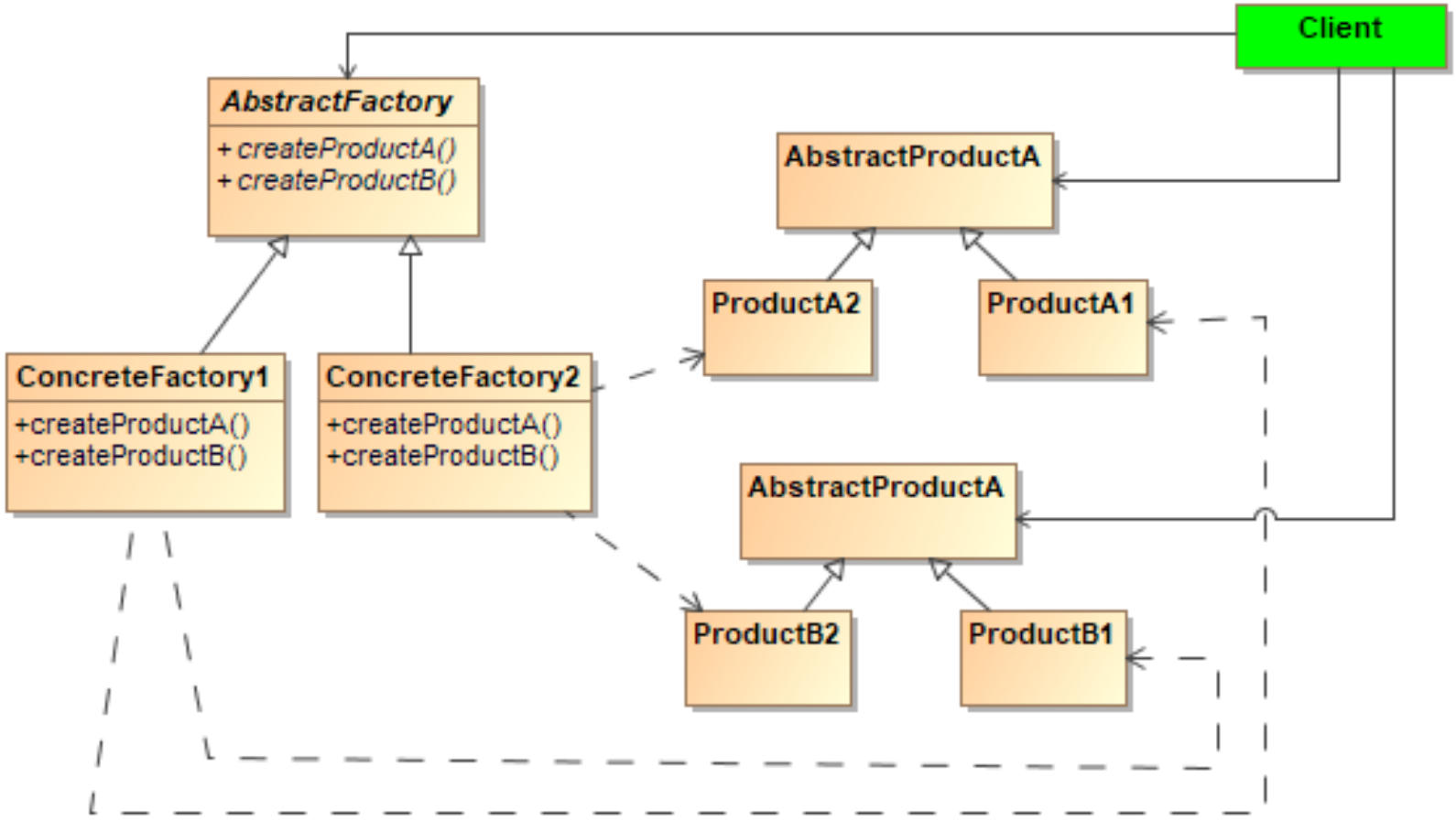
You avoid tight coupling between concrete products and client code.

Single Responsibility Principle. You can extract the product creation code into one place, making the code easier to support.

Open/Closed Principle. You can introduce new variants of products without breaking existing client code.

The code may become more complicated than it should be, since a lot of new interfaces and classes are introduced along with the pattern.

Abstract Factory structure (GoF)



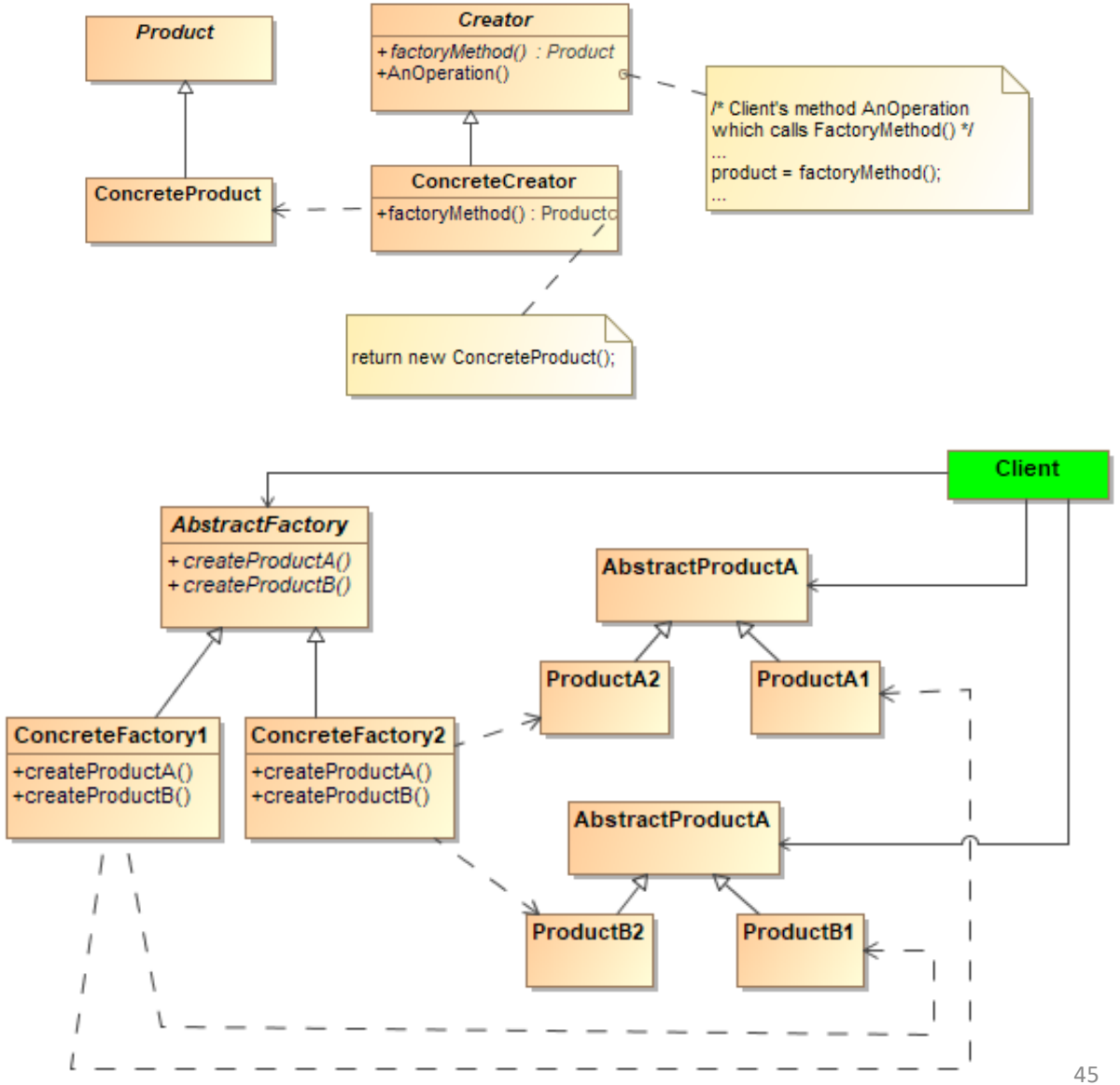
- The abstract factory is an object that has multiple factory methods on it.

- **Factory method**

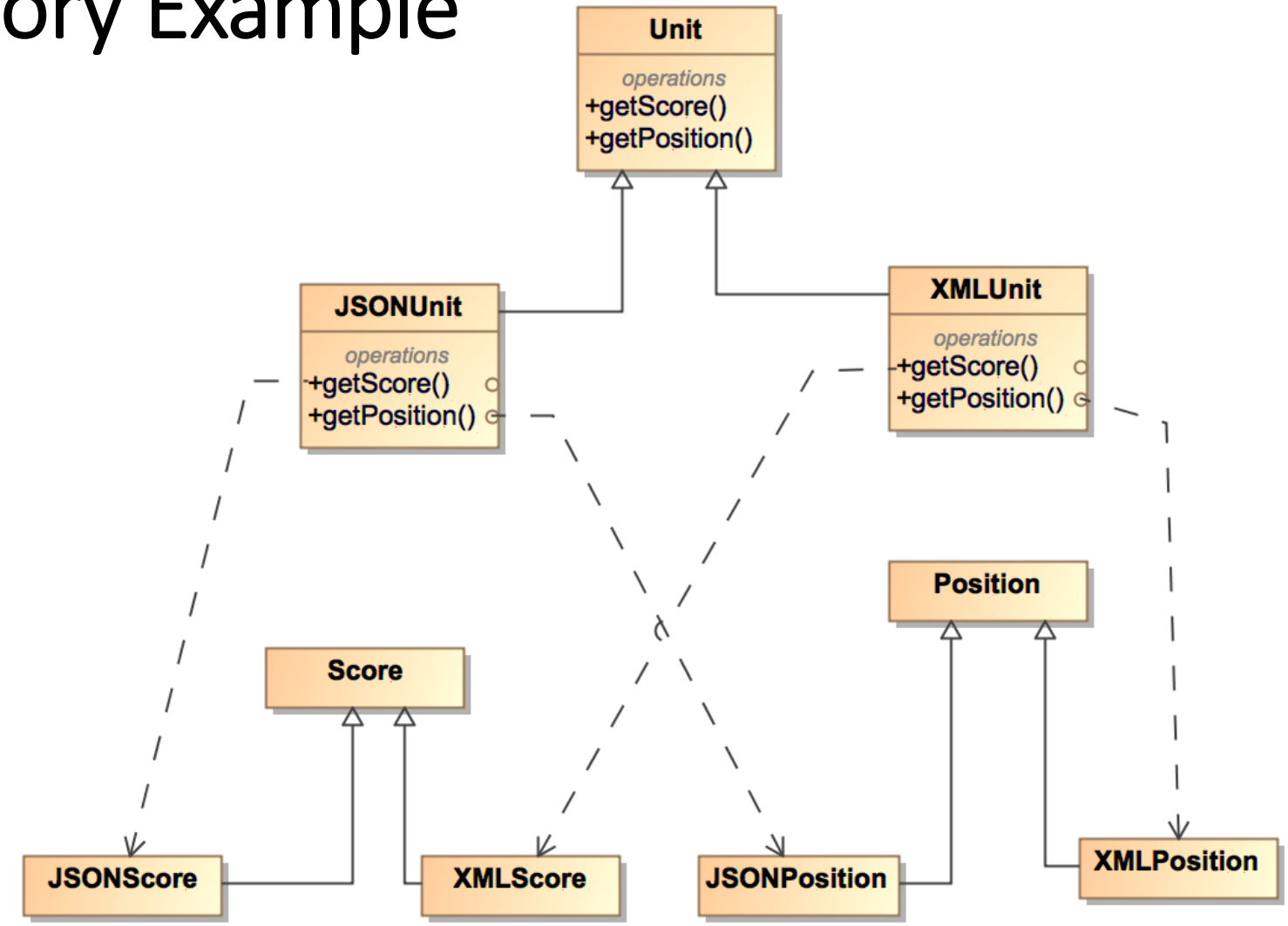
"Define an **interface** for creating an object, but let subclasses decide which class to instantiate. Factory method lets a class defer instantiation to subclasses."

- **Abstract Factory**

"Provide an **interface** for creating **families** of related or dependent objects without specifying their concrete classes"



Abstract Factory Example

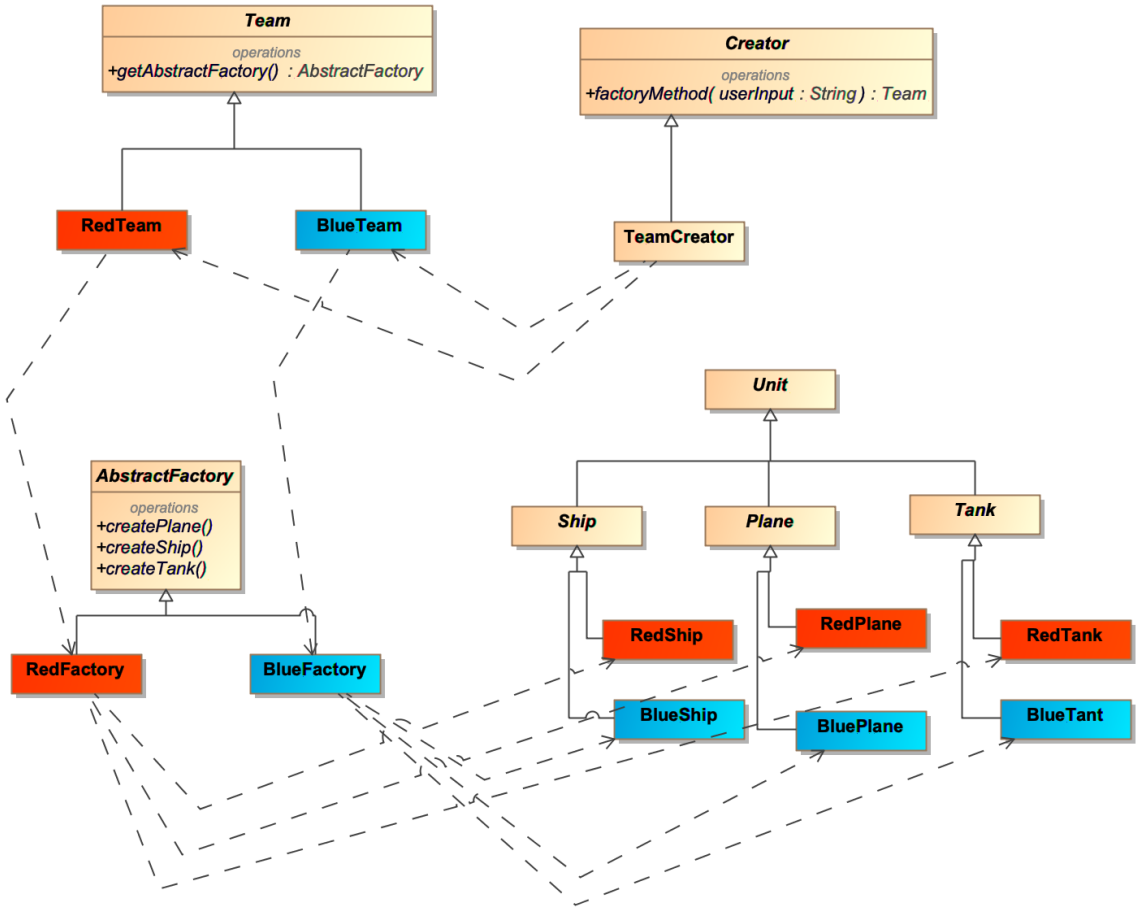


Abstract Factory example to code

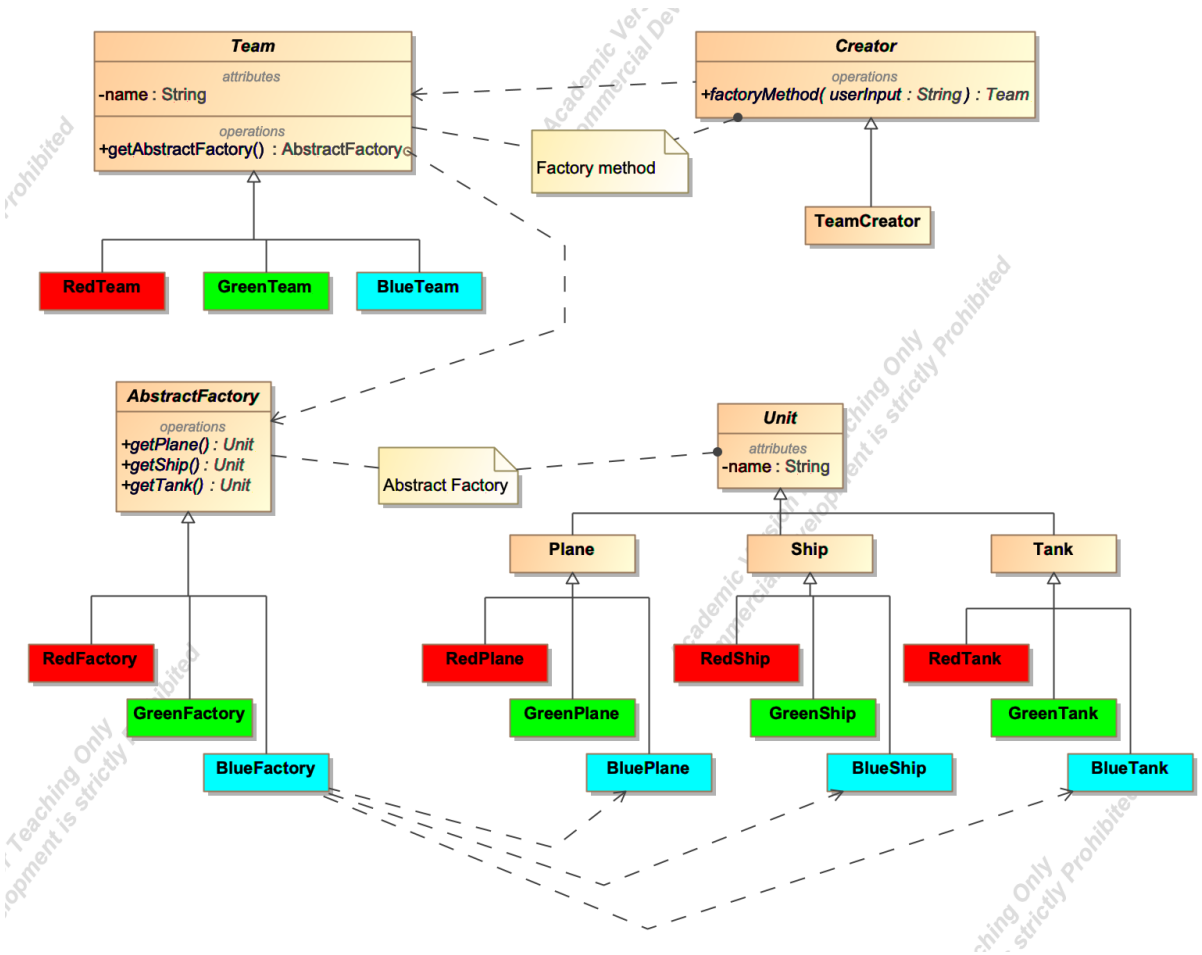
- Collection of units for each team

Factory method and Abstract factory example

- UML diagrams



Created during lecture



Rytoj pridedame Yellow Team. Kiek vietų reikės keisti?

- Nesunkiai pridedame naują komandą, ir unitus, bet kliento logikos nereikia perrašyti taip stipriai, nes ji dirba per bendrus tipus ir fabrikus.

Kas geriau: 5 labai protingos klasės ar 20 labai paprastų?

- „O dabar pažiūrėkime į mūsų main. Kiek jis žino apie visas tas 20 klasių?“
- Nubraižius modelį atrodo: „siaubas, kiek klasių“. Bet kai parašai kliento kodą, paaiškėja, kad klientas žino labai nedaug.